Final R Package Report

`peRidial` R package: Development Process and Reflections

Ernest Caballero / ID# 300353807

20 October 2026

1 Background and Development Process

`peRidial` is a standardised R framework for ingesting, validating and summarising clinical peritoneal dialysis (PD) unit data in New Zealand context. It uses nested S3 objects to support the reproducible calculation of the two ISPD peritonitis indicators, the peritonitis rate and the peritonitis-free percentage (Appendix A). That aim has not changed since the design proposal. What changed is how raw unit data reaches the objects, and what the unit object reports once it is built. The unit level methods like plot and summary are now called only by unit object and patient object. Print methods are still called by all core objects. The GitHub repository is named `peRidial`; the package itself is installed from it and loaded as `peridial`.

The package started from a four-class design proposal into a peer-tested fully operational package using mainly base R and a C++ compiled code. The time-frame from creating the package in RStudio to completing this report was between 13 August and 19 October 2026. The object model is the one proposed: `pd_unit` owns `pd_patient`, which owns `pd_catheter`, which owns `pd_infection`, and each class follows the same constructor / validator / helper split. Development ran in three phases.

Design (July to mid-August). The design proposal of 26 July fixed the four-class object model, the indicator definitions, the two input forms and the decision to use synthetic data only. Writing the formulas down before any code was the most useful step in this phase: `catheter_exposure_days()` is a direct transcription of the time-at-risk equation in Appendix A and has not changed since.

Build (mid-August to late September). The package was created with `usethis`, documented with `roxygen2`, tested with `testthat` edition 3 and version-controlled with git and GitHub. Objects were built bottom-up, infection then catheter then patient then unit, each on its own feature branch, so every object reaching a user had already passed its own validator and those of everything nested inside it. The design plan of 6 September recorded the changes made so far, added a class diagram, and tightened the cohort definition so that a positive patient count guarantees a positive patient-years denominator, which removes the division-by-zero case from both indicators.

More building and polishing (late September to 19 October). This phase absorbed most of the work: 136 of the 199 commits (68%) were made after 5 October. The ingest was split into `ingest_read`, `ingest_build`, `ingest_tables`, `ingest_tau` and `keyword_map`, and gained the data-quality issue log. `value_maps` was added to handle inconsistently recorded organism names, and input was widened to CSV and in-memory data frames. `peritonitis_rate_by_period()` and `monthly_peritonitis_rate()` were added to feed the `plot()` method, alongside the `summary()` and `subset()` methods, and three vignettes and the help pages were written. A simplified formula for peritonitis rate and peritonitis-free percentage was adopted since the proposal (see package

Vignettes).

What changed from the proposal.

1. **Relapsing episodes are not counted as new episodes.** On review of the ISPD 2022 guideline, a relapse is a continuation of the preceding episode rather than a new one (Li et al. 2022), so Equation (3) in Appendix A counts countable episodes only. The proposal counted every episode, which raises the numerator and overstates the peritonitis rate.
2. **The compiled component was retargeted.** The proposal named two candidates for C++, the exposure calculation and the episode-type classification, and only the latter was implemented. The exposure calculation, Equation (1), is arithmetic that vectorises in base R, whereas classification is sequential within a patient. Section 3.1 gives the benchmark.
3. **An episode no longer carries a catheter key.** The raw PE form does not record which catheter was in place, so each episode is matched at ingest to the catheter whose PD window contains its infection date.
4. **The censoring date τ is derived rather than supplied.** Four competing events can end a patient's time at risk (death, transplantation, permanent transfer to haemodialysis and, by default, PD stopping with no successor catheter), and the earliest is taken. The last is controlled by `censor_on_last_stop`, since a genuine break from PD looks identical to a permanent exit in the raw data.
5. **Data-quality problems are collected and reported together.** Rather than aborting on the first problem, the ingest records each issue it finds, with the patient, catheter or file row concerned, and reports them in one pass. Problems that make a unit invalid are raised as a single error listing all of them, so no unit is built; recoverable problems are recorded as warnings. A unit with many problems may still need several runs.
6. **The input contract needs to align with the package.** A PD-specific national registry exists but is not publicly accessible (New Zealand Peritoneal Dialysis Registry 2024), while the ANZDATA registry is public and records overlapping items. A raw extract from either registry would need preprocessing before the package could use it: the required columns must be selected and coded values decoded. The ingest then matches columns by keyword against the required columns, and accepts Excel, CSV or in-memory data frames.

There are several smaller departures that can be noted. The per-class `peritonitis_rate()` and `peritonitis_free()` functions in the proposal were replaced by `summary()`, `count_episodes()`, `total_patient_years()` and the period-rate functions, because a rate cannot be interpreted without its numerator, denominator and benchmark beside it. `summary()` exists for two classes rather than four, and `subset()` returns a flat table rather than a pruned object, so that counts are never recomputed. The example data ship as four Excel files in `inst/extdata`, a clean pair and a deliberately malformed pair, so that the real entry point is exercised; the cost is that no seeded script exists to regenerate them. Height and weight columns are not utilised in any way but are present in the raw files.

2 The Final Package

All figures are measured from the repository at commit `17b125c` (19 October 2026).

Component	Contents
R code	5,305 lines in 17 files

Component	Contents
Compiled code	72 lines of C++ (`src/episode_type.cpp`). `classify_episode_types()` can run an equivalent pure-R loop with `use_cpp = FALSE`; `pd_unit()` always uses the C++
Interface	14 exported functions and 10 S3 methods (`print` for each class, `summary` for two, `subset` for three, `plot` for the unit)
Tests	13 test files with over 500 expectations
Documentation	25 help pages and 3 vignettes
Example data	Four synthetic Excel files: a clean A3/PE pair and a deliberately malformed pair

A user supplies the unit's two raw files (the A3 form, one row per catheter, and the Peritonitis Episode (PE) form, one row per infection), a reporting period and a unit identifier. `pd_unit()` returns a validated unit, or stops with a single error listing the data problems found in that pass. `summary()` then reports the rate and the peritonitis-free percentage against their ISPD benchmarks with the numerator and denominator of each, an episode-type breakdown and the incident/prevalent split; `plot()` draws the unit's reporting panels; and `subset()` returns a filtered flat table without recalculating counts.

2.1 End-user

The intended user is the person in a New Zealand PD unit who is responsible for compiling the unit's collected data, usually an administrator or a clinician. This person validates the unit records before they are submitted to the national coordinator, or entered into a database accessed by the person in charge of the ANZDATA submission.

What this user needs is a way to turn the unit's own records into the required columns, and to know where those records fall short. Unit spreadsheets rarely share the registry's column names or coding, so the package preprocesses the input: it matches columns by keyword, decodes values, accepts Excel, CSV or in-memory data frames, and reports every gap in the issue log so that it can be corrected at the source. The rate, the peritonitis-free percentage and the reporting plots are then produced from the validated data rather than calculated by hand.

Future work. The package still requires R, which excludes users who are not comfortable with code, such as the nurses who collect the data on the unit. A natural next step is an R Shiny app that wraps `pd_unit()`, `summary()` and `plot()`, in which a user uploads the two forms, sees the issue list and reads the indicators. This would improve data collection practice, because problems would be visible to the people who enter the data, and would reduce the gaps in reporting that come from manual calculation. It would also allow a unit to review its rates, which `peritonitis_rate_by_period()` already returns by period, whenever the data are updated, so that a rise in the peritonitis rate, or a drop in the peritonitis-free percentage, is noticed early rather than at the yearly report.

3 Reflections and Self-critique

3.1 What worked

Clinical knowledge of the problem. I came to this project as a dialysis nurse who has collected and transcribed these records by hand. That shaped the decisions regarding: time at risk starts when PD starts, not at catheter insertion; a relapse is not a new episode; an `NA` outcome means

the episode resolved, not that the value is missing; a stop in PD could mean a genuine gap in PD or a missing entry. From that experience, a package that reads forms modelled against real data, calculates the indicators, produces plots and names every gap in the data would save a unit a great deal of manual work.

Nesting justified by censoring. The object graph is nested because censoring is a property of a person while exposure is a property of a catheter. `total_patient_years()` therefore takes the nested `patient_list` rather than the flat catheter table, and the patient-level transfer date is applied to every catheter below it. Flattening everything to tibbles would have meant re-establishing that relationship with a join at every call site.

One place for the counting rule. `count_episodes()` applies both filters: the intersection of the catheter's PD window with the reporting period, and the exclusion of relapsing episodes. When the relapse definition was corrected, one function changed and the classes above it inherited the correction.

A data-quality issue log. Every problem found at ingest is recorded with the patient, catheter or file row concerned and reported together. This diagnoses problems in registry inputs before they reach an indicator, so the reported figures represent the data more accurately, and it addresses the gap, noted in NZPDR reporting, that missing data needs to be treated as a priority.

Demographics and plots in the summary. The summary reports patient-level demographics: type of PD, primary kidney disease, procedure type, organisms cultured and outcome. Following my supervisor's advice that the package should support routine reporting, these values also drive the plots, so the package can reproduce the kind of figures the NZPDR report shows (New Zealand Peritoneal Dialysis Registry 2024) in addition to the rate and the peritonitis-free percentage.

Compiling the episode classifier rather than the exposure calculation. The proposal named two candidates for C++: the exposure calculation and the episode-type classification. The classification was the better target. It is sequential within a patient, because each episode depends on the preceding episode's organisms and antibiotic completion date, so it is naturally written as a loop; the exposure calculation, by contrast, vectorises in base R. Benchmarked on 20,000 simulated episodes (2,000 patients with 10 episodes each), the C++ version took 0.64 s against 1.41 s for the equivalent R loop, about 2.2 times faster, with identical results. The gain is limited because each row is still validated in R in both modes. The compiled code also has a cost: because `pd_unit()` always uses it, installing from source needs a compiler toolchain (Rtools on Windows, the Xcode command line tools on macOS), which may be an obstacle on some managed hospital machines. However, given the size of cohort of NZ PD units, a simple base R would do just fine. The C++ component will outperform base R when simulation studies are done.

3.2 What did not work

3.2.1 The input data was modelled rather than obtained

I had no access to real unit exports. The NZPDR data are restricted, and the only public data dictionary is ANZDATA's (ANZORRG 2025). The synthetic A3 and PE columns were modelled on the ANZDATA definitions for patient- and episode-level items and, from memory, on the NZPDR for catheter-level items such as procedure type and removal reason. The column names themselves are my own, chosen to align best with the NZPDR. Real exports can also mix coded and decoded values. The remedy was to identify the columns the indicators actually require and match them by keyword (`keyword_map`), which made preprocessing the largest hidden cost of the project. Many hours went into edge cases, making sure data is ingested correctly and that any gap is warned about and reported, because standardising reporting and validating clinical data is the purpose of

the package.

The ingest also has limits that a user must know, and the vignettes document them. A misnamed optional column is filled with `NA` without a warning, so a misnamed Date of Death leaves patients uncensored and overstates patient-years. The infection file must include episodes from at least 28 days before `t0`, or a relapse in the first weeks of the period is counted as new and the rate is overstated; `pd_unit()` warns only when this may have happened. Patient IDs must be three letters followed by four digits following NZ national Health Index number format, so an ANZDATA extract, which identifies patients by an integer registry ID, fails the check even after its columns are renamed.

3.2.2 Clinical definitions were coded before they were pinned down

I built `pd_infection`, `pd_catheter` and `pd_patient`, and tests for all three, on a reading of the ISPD guideline that counted relapsing episodes in the numerator. The guideline does not count them (Li et al. 2022). The error went unnoticed for about three weeks, until I reread the guideline. Correcting it meant changing the numerator, the per-catheter and per-patient counts, their tests, and the associated documentation. A counting function addressed this but writing the rule down and testing it against the guideline before building the classes would have avoided it.

3.2.3 Two views of the same data must be kept consistent

`pd_unit` holds both a nested `patient_list` and three flat tibbles derived from it. Keeping the two views consistent is extra work: `validate_pd_unit()` has to check that `nrow(patients)`, `length(patient_list)` and `n_patients` agree and name the same people. Those checks exist only because the design stores the same information twice. The tibbles were used mainly for subset methods.

3.2.4 A unit cannot be built by hand

The infection, catheter and patient objects can each be built by hand and nested, `pd_infection` inside `pd_catheter` inside `pd_patient`, but a unit cannot be built from them. `pd_unit()` accepts only the two input tables. The exported `new_pd_unit()` takes the headline counts, patient-years and flat tables as arguments instead of deriving them from a list of patients, so a user would have to calculate all of them and the result would not be validated. In practice a unit object is created only by the ingest. I considered adding a function that assembles and validates a unit from a list of `pd_patient` objects, which would let a unit be built without raw input files, but I ran out of time.

3.2.5 Testing grew and overlapped

The 13 test files now hold over 500 expectations, but two weaknesses remain. First, some tests are redundant, because I did not draw the line in advance between tests of an object, tests of an individual function, and tests that confirm the parent-child relationships in the nested objects. Second, direct coverage has gaps: the tests for `exposure_days()` and `catheter_exposure_days()` were added only after the core objects, `count_patient_episodes()` still has no isolated test, and the flattening helpers are reached only through `subset()`. By carefully planning unit testing, I would have avoided making tests that was covered in other functions.

3.3 Changes I would make

1. **Nail the definitions, then write the tests from them.** I would have started with nailing the definitions, ISPD rules, its formula and worked examples, including the 4-week boundary and the culture-negative pairings, then a test file written from that, then the classes. The

relapse error would have surfaced before any class existed, and each rule would have one 'truth' statement.

2. **Start from the data dictionary, with a messy fixture.** Build the keyword matching and a deliberately malformed example file in the first weeks, using the ANZDATA dictionary and a request to the NZPDR for a column list or sample export, and let the object fields follow what the forms contain. The ingest absorbed the most unplanned work because its contract was not matched with the actual exported columns.
3. **Write the quarterly report as a script first.** Design the public functions backwards from the report the package has to produce. The functions that proved most useful (`peritonitis_rate_by_period()`, `plot()` and `subset()`) were late additions, while the per-class indicator functions specified in the proposal were never exported.
4. **Benchmark before compiling.** Write the loop in R, measure it on realistic and on larger inputs, and compile only if the gain justifies the cost of having an extra tool.
5. **Keep one source of truth.** Store the nested tree only and generate the flat tables on demand. This would remove the consistency checks between the two objects in `validate_pd_unit()`.
6. **Build a unit from its patients.** Give `pd_unit()` a second route that takes a list of validated `pd_patient` objects and derives the counts, patient-years and summary tables itself, rather than taking them as arguments as `new_pd_unit()` does now. Every level of the hierarchy could then be built from the level below it, as the infection, catheter and patient objects already are, and tests could build small units by hand instead of relying on Excel fixtures.

4 References

ANZORRG. 2025. *ANZDATA Data Dictionary, Version 2025.3*. ANZORRG Registry. https://anzorrg.org.au/data-management/data-dictionary.

Li, Philip Kam-Tao, Kai Ming Chow, Yeoungjee Cho, et al. 2022. "ISPD Peritonitis Guideline Recommendations: 2022 Update on Prevention and Treatment." *Peritoneal Dialysis International* 42 (2): 110–53. https://doi.org/10.1177/08968608221080586.

New Zealand Peritoneal Dialysis Registry. 2024. *New Zealand Peritoneal Dialysis Registry (NZPDR) Annual Report 2024: A Comprehensive Overview of Data, Analysis, and Insights for the Year*. New Zealand Peritoneal Dialysis Registry.

A Indicator Definitions

A.1 Time at risk

$$d = \max\Big(0,\ \min(\text{stop},\, t_1,\, \tau) - \max(\text{start},\, t_0) + 1\Big) \tag{1}$$

where

- d is the catheter's days at risk in the period;
- start and stop are the dates PD started and stopped through that catheter. start is the start of time at risk, not the catheter insertion date. If there is no stop date the catheter is still in use (`NA`), and stop is left out so that t_1 closes the interval;
- t_0 and t_1 are the first and last day of the reporting period, both included;
- τ is the date the patient permanently left PD: death, transplant, permanent transfer to haemodialysis (HD) or the last catheter closing with no successor catheter.

A.2 Total patient-years at risk

$$PY = \frac{\sum d}{365.25} \tag{2}$$

where the sum runs over every catheter of every patient in the cohort, and PY is the unit's total patient-years at risk. The same sum over one patient's catheters is that patient's time at risk.

A.3 Peritonitis rate

$$\text{rate} = \frac{E}{PY} \qquad \text{(episodes per patient-year)} \tag{3}$$

where E is the number of *countable* episodes in the cohort and PY is as above. An episode is countable only if:

1. its infection date lies inside the active window of the catheter it belongs to, which is $[\max(\text{start}, t_0),\ \min(\text{stop}, t_1, \tau)]$, the same dates that bound the exposure above, so the numerator and the patient-years cover the same time; and
2. it is not a *relapsing* episode.

A.4 Peritonitis-free percentage

$$\text{PF} = \frac{N_0}{N} \tag{4}$$

where N is the number of patients in the cohort and N_0 is how many of them had no countable episode in the period. PF is expressed as a proportion here and multiplied by 100 for the percentage. The ISPD benchmarks are ≤ 0.40 episodes per patient-year and PF > 0.80 (Li et al. 2022).